The medium

This is a small language for describing computations. We will build it around one game, rock, paper, scissors: choosing hands, playing rounds, keeping score, and asking a person to play.

Each example introduces a piece or shows how pieces combine. Alongside them, a few lines of notation keep track of the shapes. By the end, those lines fit together into one map.

1. Things

First, we need the game's three hands:

```
Hand = rock | paper | scissors
```

This reads: a hand is rock, paper or scissors. `Hand` is a **type**: it tells us what kind of thing we mean. The names on the right are **constructors**. For now, each names a different thing.

A round also has an outcome:

```
Outcome = win | lose | draw
```

A hand is one of three choices. An outcome is one of three results. The names change; the shape stays the same:

```
T = c | …
```

`T` stands for a type's name, and `c` for a constructor. `| …` means we can list more constructors. These letters describe the shape; they aren't names in the game.

Each type is declared once, and each constructor belongs to one declaration. Types start with a capital letter; constructors don't.

2. Functions

Paper beats rock, scissors beat paper, and rock beats scissors. We can put those three answers into one rule:

```
better : (Hand) → Hand
better = (rock) ↦ paper
       | (paper) ↦ scissors
       | (scissors) ↦ rock
```

`better` is a **function** with three **cases**, separated by `|`. Each case is what it is given, followed by what it gives back. `↦` reads “gives”.

The line above the definition gives its **function type**. It reads “better goes from a hand to a hand”. `→` describes the kinds of inputs and results; `↦` gives the answer in a particular case.

To use the function, put an input after its name:

```
better(rock)
    becomes   paper
```

This is **applying** the function. We find the first case that matches the input, and replace the application with that case's answer. A constructor matches itself: rock matches rock.

`better(rock)` and `paper` are both **expressions**: pieces of the language. But `paper` has nothing left to work out. It is already a **value**.

The input can itself be something to work out:

```
better(better(rock))
    becomes   better(paper)
    becomes   scissors
```

The inner application goes first. Once it gives us `paper`, the outer one can use it. This is how we combine functions: the result of one becomes the input of another. The type helps us see why this combination works: `better` gives back the same kind of thing it takes.

Suppose we want the hand that rock beats. That is scissors: the result of using `better` twice. The same works for every hand, so let's give it a name:

```
worse : (Hand) → Hand
worse = (a) ↦ better(better(a))
```

`a` is a **variable**: it wasn't declared as a constructor, so in a pattern it matches any value and names that value inside its case. Given `paper`, we put `paper` wherever this `a` occurs:

```
worse(paper)
    becomes   better(better(paper))
    becomes   better(scissors)
    becomes   rock
```

The part before `↦` is the **pattern**: it matches the input. The expression after `↦` is the **body**. The body waits until we apply the function, and only the case we use is worked out.

So a definition has this shape:

```
n = (p) ↦ e | …
```

`n` is the function's name, `p` a pattern, and `e` an expression. The body can be a value, as in `better`, or an application, as in `worse`.

Matching can fail. `better(win)` gets **stuck**: `win` is a value, but none of the cases accepts it. The type line helps us see the mismatch. Writing a type doesn't change how the cases work.

Several inputs

To find who wins a round, we need two hands:

```
result : (Hand, Hand) → Outcome
result = (rock, scissors) ↦ win
       | (scissors, paper) ↦ win
       | (paper, rock) ↦ win
       | (rock, rock) ↦ draw
       | (paper, paper) ↦ draw
       | (scissors, scissors) ↦ draw
       | (a, b) ↦ lose
```

Each input matches its own pattern. The first three cases are wins for the first player; the next three are draws. The last case accepts anything left over, so it means “otherwise”.

```
result(paper, rock)       becomes   win
result(rock, paper)       becomes   lose
```

Several inputs are worked out from left to right:

```
result(better(rock), better(paper))
    becomes   result(paper, better(paper))
    becomes   result(paper, scissors)
    becomes   lose
```

A function can take any number of inputs, even none:

```
later : () → Hand
later = () ↦ better(rock)

later()
    becomes   better(rock)
    becomes   paper
```

With no inputs, we still have a body waiting to be used. Its application is what starts it.

The shapes now allow several patterns and several input types:

```
n : (A, …) → B
n = (p, …) ↦ e | …
```

`A` and `B` stand for types. A comma followed by … means any number of these places, including none. Every case of one function has the same number of patterns, and the function takes that many inputs.

The variables in one case must have different names. In `(a, a)`, the same name would be given two inputs. To match two rocks, write `(rock, rock)`, as above.

3. Data and patterns

A round needs two hands. We can keep them together:

```
Round = round(Hand, Hand)
```

A `Round` is made with `round`, holding two hands:

```
round(paper, rock)
```

The first player chose paper, and the second chose rock. This is already a value. The constructor holds the two hands together. Values built with constructors are **data**.

To use those hands, write the same shape as a pattern:

```
judge : (Round) → Outcome
judge = (round(a, b)) ↦ result(a, b)

judge(round(paper, rock))
    becomes   result(paper, rock)
    becomes   win
```

`round(a, b)` takes the round apart: `a` names its first hand, and `b` its second. A body can put data together again:

```
flip : (Round) → Round
flip = (round(a, b)) ↦ round(b, a)

flip(round(paper, rock))
    becomes   round(rock, paper)
```

The two uses have related shapes:

```
c(e, …)       build data from expressions
c(p, …)       match data with patterns
```

There is one place for each thing the constructor holds. A constructor that holds nothing, like `rock`, is written without parentheses.

Each `e` can be an expression. We work out the contents from left to right before the data is a value:

```
round(better(rock), rock)
    becomes   round(paper, rock)
```

Each `p` can be a pattern. `round(rock, a)` fixes the first hand and leaves the second open. It matches `round(rock, paper)`, with `paper` as `a`, but doesn't match `round(paper, rock)`. `round(a, b)` leaves both hands open.

A type describes which contents fit. `round(win, lose)` can still be written: it is data, but it isn't a `Round`.

4. Functions as values

We used `better` twice to define `worse`. We can do the same with any rule from hands to hands:

```
twice : ((Hand) → Hand, Hand) → Hand
twice = (f, a) ↦ f(f(a))
```

Read its type from the outside: two inputs, then one result. The first input is a function from hands to hands. The second is a hand. The result is a hand.

```
twice(better, rock)
    becomes   better(better(rock))
    becomes   better(paper)
    becomes   scissors
```

`better` on its own is the function. `better(rock)` applies it. A function is a value, so we can pass it around before using it.

We can also write a function directly, without giving it a name:

```
(a) ↦ rock
```

This rule accepts an input and always gives back rock. It can go wherever a function can go:

```
twice((a) ↦ rock, paper)
    becomes   ((a) ↦ rock)(((a) ↦ rock)(paper))
    becomes   ((a) ↦ rock)(rock)
    becomes   rock
```

Parentheses keep each function together. A body extends as far to the right as it can, so we group a function when something after it belongs outside its body.

Giving back a function

Suppose we want to choose the hand that a rule always gives back:

```
always : (Hand) → (Hand) → Hand
always = (a) ↦ (b) ↦ a
```

Given a hand, `always` makes a rule that ignores its own input and gives back that hand:

```
always(rock)
    becomes   (b) ↦ rock
```

The outer input went into the inner function's body, where `a` was. That body still waits until someone applies the new function:

```
always(rock)(paper)
    becomes   ((b) ↦ rock)(paper)
    becomes   rock
```

The type makes the two stages visible. `(Hand) → (Hand) → Hand` means “from a hand to a function from hands to hands”. What follows `→` stays together, just as a body does. Two stages, `f(a)(b)`, are different from giving one function two inputs, `f(a, b)`.

A function can take a rule and make another rule from it:

```
twofold : ((Hand) → Hand) → (Hand) → Hand
twofold = (f) ↦ (a) ↦ f(f(a))

twofold(better)
    becomes   (a) ↦ better(better(a))
```

That is the rule used by `worse`. And the rule made by `always` can be passed to `twice`:

```
twice(always(rock), paper)
    comes to   rock
```

“Becomes” shows one step. “Comes to” skips over one or more steps to show where they lead.

Keeping names clear

A variable belongs to its case. Functions inside the body can use it, as `always` uses `a`. If an inner case introduces the same name, that inner name takes over there:

```
((a) ↦ (a) ↦ a)(rock)(paper)
    becomes   ((a) ↦ a)(paper)
    becomes   paper
```

The last `a` belongs to the inner case, so rock never replaces it. We replace a variable only where its name means that variable.

Variables cannot reuse a constructor's or a defined function's name. Each global name has one meaning, and each function is defined once. Definitions may refer to one another regardless of their order; we will use that when we reach repetition.

Holding a function

A player can hold its rule:

```
Player = player((Hand) → Hand)

player(better)
player(always(rock))
```

The first player chooses the hand that beats the one it is given. The second always chooses rock. A pattern gets the rule out, and application uses it:

```
choose : (Player, Hand) → Hand
choose = (player(f), a) ↦ f(a)

choose(player(better), rock)
    becomes   better(rock)
    becomes   paper
```

A pattern can name a function, as `f` does here, but can't inspect its cases. We use a function by applying it.

An application has this shape:

```
e(e, …)
```

First work out the expression on the left to find the function. Then work out its inputs, from left to right, and apply it. In `always(rock)(paper)`, finding the function means working out `always(rock)` first.

If the value found isn't a function, or takes a different number of inputs, the application gets stuck:

```
twice(rock, paper)
    becomes   rock(rock(paper))            stuck: rock isn't a function
twice(result, rock)
    becomes   result(result(rock))         stuck: result takes two inputs
```

These mistakes arrived through `f`. A name written directly must be given the right number of inputs; passing it through a variable can expose the mismatch only when we apply it.

5. Repeating

To keep score, we need numbers. We can make them from two constructors:

```
Nat = zero | succ(Nat)
```

`succ` means “one more than”. Zero is a number; one more than a number is another number. So `zero`, `succ(zero)` and `succ(succ(zero))` are zero, one and two.

The declaration uses its own name. A function can do that too:

```
add : (Nat, Nat) → Nat
add = (zero, b) ↦ b
    | (succ(a), b) ↦ succ(add(a, b))
```

Zero and `b` make `b`. One more than `a`, and `b`, make one more than `a` and `b`:

```
add(succ(succ(zero)), succ(zero))
    becomes   succ(add(succ(zero), succ(zero)))
    becomes   succ(succ(add(zero, succ(zero))))
    becomes   succ(succ(succ(zero)))
```

Each step takes a layer off the first input and puts it around the result. The same two cases work however many layers there are. This is **recursion**: using a name inside its own definition.

A win earns one point:

```
points : (Outcome) → Nat
points = (win) ↦ succ(zero)
       | (other) ↦ zero

points(draw)       becomes   zero
```

Here the last case overlaps the first: it could match win too. Trying cases in order is what makes win worth a point and everything else worth zero.

Recursion needn't make progress:

```
loop : (Hand) → Hand
loop = (a) ↦ loop(a)

loop(rock)
    becomes   loop(rock), and so on forever
```

A function type doesn't promise that a computation finishes. `(Hand) → Hand` says that on hand inputs it doesn't get stuck, and if it finishes, its result is a hand. `loop` never finishes. By contrast, `(rock) ↦ paper` isn't of this type: it gets stuck on paper or scissors.

So far, working out an expression can finish with a value, get stuck, or go on forever.

6. Leaving a type open

A match is a list of rounds. The hands a player has shown are a list too. We can describe both with one declaration:

```
List(Item) = empty | cons(Item, List(Item))
```

`Item` leaves the type of each item open. A `List(Hand)` is empty, or a hand followed by a list of hands. A `List(Round)` has the same shape with rounds in those places.

```
cons(rock, cons(paper, empty))
```

This is a list of two hands. `cons` holds the first item and the remaining list. A function can follow that structure:

```
size : (List(Item)) → Nat
size = (empty) ↦ zero
     | (cons(a, rest)) ↦ succ(size(rest))

size(cons(rock, cons(paper, empty)))
    comes to   succ(succ(zero))
```

In a signature, an undeclared capitalised name such as `Item`, written where a type goes, stands for an arbitrary type. Use the same choice everywhere that name occurs in that signature. The signature

describes the function for every choice: `size` works for lists of hands, rounds, or any other kind of value. Types guide our reading; they aren't extra inputs to the function.

We can transform every item in a list:

```
each : ((From) → To, List(From)) → List(To)
each = (f, empty) ↦ empty
     | (f, cons(a, rest)) ↦ cons(f(a), each(f, rest))

each(better, cons(rock, cons(paper, empty)))
    comes to   cons(paper, cons(scissors, empty))
each(points, cons(win, cons(draw, empty)))
    comes to   cons(succ(zero), cons(zero, empty))
```

The first example changes hands into hands. The second changes outcomes into numbers. The signature shows the connection: the function takes the kind of item in the input list and gives the kind in the output list.

Two independent types can be left open too:

```
Pair(First, Second) = pair(First, Second)
```

`pair(win, zero)` is a `Pair(Outcome, Nat)`. The declaration's type variables belong only to that declaration. Their names must be different, and must not reuse a declared type's name. Types can nest: `List(List(Hand))` is a list of lists, and `List((Hand) → Hand)` is a list of rules.

The declaration shape is now:

```
T(X, …) = c(A, …) | …
```

`X` stands for a type variable. Each `A` is a type, possibly using those variables. A type is filled with as many types as its declaration takes. With none, write `Hand`, without parentheses. The same convention gives us `rock` for a constructor with no contents.

Playing a match

We can now let two rules play. Each rule sees the other player's previous hand. `play` asks both for their next hands; `meet` puts those hands into a round and plays the remaining rounds:

```
play : (Nat, (Hand) → Hand, (Hand) → Hand, Hand, Hand) → List(Round)
play = (zero, f, g, a, b) ↦ empty
     | (succ(left), f, g, a, b) ↦ meet(left, f, g, f(b), g(a))

meet : (Nat, (Hand) → Hand, (Hand) → Hand, Hand, Hand) → List(Round)
meet = (left, f, g, a, b) ↦ cons(round(a, b), play(left, f, g, a, b))
```

The two definitions use each other. Two rounds of `better` against “always rock”, starting with rock as both previous hands:

```
play(succ(succ(zero)), better, always(rock), rock, rock)
    comes to   cons(round(paper, rock), cons(round(paper, rock), empty))
```

And we can count the wins:

```
wins : (List(Round)) → Nat
wins = (empty) ↦ zero
     | (cons(this, rest)) ↦ add(points(judge(this)), wins(rest))

wins(play(succ(succ(zero)), better, always(rock), rock, rock))
    comes to   succ(succ(zero))
```

7. The core together

Every value we have made is data or a function:

A value	How we make it	How we use it
Data	A constructor with its contents	A pattern that takes it apart
A function	Cases with patterns and bodies	Application to inputs

Contents, inputs and results can themselves be data or functions. A type describes which kind belongs in each place.

We can collect the shapes of expressions in one place. `::=` reads “has one of these shapes”:

```
e ::= x                    a variable
      c(e, …)              data
      n                    a defined name
      (p, …) ↦ e | …       a function
      e(e, …)              an application
```

`x` stands for a variable; the other letters are the ones we have used above. Each place marked `e` can hold another expression. A function can have a function as its body, as in `always`. A constructor can hold an application, as in `player(always(rock))`. An application can find its function through another application, as in `always(rock)(paper)`.

Patterns also fit inside patterns:

```
p ::= x                    name the whole value
      c(p, …)              match its constructor and contents
```

For example, `cons(round(rock, a), rest)` looks inside both the list and its first round. It matches `cons(round(rock, paper), empty)`, with paper as `a` and the empty list as `rest`.

Types have three shapes:

```
A ::= T(A, …)              a declared type, filled with types
      X                    a type variable
      (A, …) → B           a function type
```

The same idea applies here. Each place for a type can hold another function type or filled type. That is how `List((Hand) → Hand)` and `((Hand) → Hand) → (Hand) → Hand` are built.

These shapes say where the pieces fit. Matching, application and the order of evaluation say what happens when we use them. A program consists of its declarations and definitions, followed by an expression to work out.

These pieces are enough to describe any computable transformation of data. Next we will let a computation ask for something while it runs: a hand someone chooses, for example.

8. Requests

Everything so far has been **saying**. Each step rewrites part of an expression into something that means the same: `worse(paper)` and `rock` are two ways of writing one hand. Computing can also be pictured another way, as steps that change a memory kept beside the program. That is **doing**. The two pictures compute the same things, but some ideas are natural only as doing: waiting for a person, keeping a score that changes, giving up partway.

The medium keeps saying underneath and brings doing back on top. It adds a new form only when an idea can't be said **locally** with the forms already there: either not at all, or only by changing parts of the program far from where the idea is used. Waiting for a person is the first such idea.

So far, we have chosen every hand in the program. To let a person choose, we need a way to ask:

```
Unit = unit

effect Console = read : () → Hand
               | show : (Outcome) → Unit
```

`Console` is an **effect**, declared by listing its **requests**. `read` takes no input and asks to be answered with a hand. `show` takes an outcome to display. Its answer is `unit`, the only value of `Unit`: there is nothing useful to give back, only something done.

An effect declaration has this shape:

```
effect F = r : (A, …) → B | …
```

`F` stands for an effect's name, and `r` for a request's name. Each request gives the types of its inputs and, after `→`, the type of its answer.

```
show(result(read(), rock))
```

This asks for a hand, finds its outcome against rock, and shows that outcome. The first thing it needs is `read()`. A request has no answer of its own, so the program waits there.

Suppose the person chooses paper. The answer goes where the request was:

```
show(result(paper, rock))
    becomes   show(win)
```

Now it waits at `show(win)`. Whatever runs the program displays win and answers `unit`. The whole expression then becomes that answer, and is finished.

These are ordinary expressions with one new form inside them:

```
r(e, …)
```

Its inputs are worked out from left to right, just like inputs to a function. Each request belongs to one effect and takes the number of inputs in its declaration.

Saying what a function may ask

We can give the game a name and keep it for later:

```
game : () → Unit ! {Console}
game = () ↦ show(result(read(), rock))
```

`! {Console}` reads "may ask Console". Calling `game()` starts the requests. Until then, the body waits like any other function's body.

A person can supply a playing rule too:

```
human : (Hand) → Hand ! {Console}
human = (previous) ↦ read()
```

The rule ignores the previous hand and asks the person for the next one. Its result is a hand, provided the request is answered with a hand.

A function type now has this shape:

```
(A, …) → B ! {F(A, …), …}
```

Some effects take types, as we will see with memory. With none, write just `Console`. With no effects, leave out the whole `! {…}` part.

"May ask" includes not asking. So `better` also fits `(Hand) → Hand ! {Console}`. We can keep rules and people in one list:

```
cons(better, cons(human, empty))
```

Its type is `List((Hand) → Hand ! {Console})`. The function type describes inputs, results, and requests that may be left for something else to answer. If those requests receive answers of their declared types, the function must avoid getting stuck and, if it finishes, give a result of its stated type.

Asking now or later

We can ask a person for a hand, then make a rule that always gives that hand:

```
pick : () → ((Hand) → Hand) ! {Console}
pick = () ↦ always(read())
```

Calling `pick()` asks once. If the person answers paper, it gives back `(b) ↦ paper`. Applying that rule asks nothing: the hand is already chosen.

Parentheses group a type just as they group an expression. Here they keep the returned rule's type, `((Hand) → Hand)`, together. The `! {Console}` outside those parentheses belongs to the call that makes the rule.

To return a rule that asks each time it is used:

```
askLater : () → ((Hand) → Hand ! {Console})
askLater = () ↦ human
```

Calling `askLater()` gives back `human` without asking. Calling the returned rule asks for a hand. The effect is inside the parentheses because it belongs to that rule. Each `! {…}` belongs to the nearest preceding arrow in the same group; without parentheses, arrows extend to the right.

Leaving the requests open

`each` applies a function to every item in a list. That function can be `human` too:

```
each(human, cons(rock, cons(paper, empty)))
```

This asks for two hands, from left to right, and collects the answers. If the person answers scissors and then rock, the result is `cons(scissors, cons(rock, empty))`.

The earlier signature described `each` with a function that asks nothing. We can give the same definition a wider description:

```
each : ((From) → To ! {Asks}, List(From)) → List(To) ! {Asks}
```

`Asks` leaves a list of effects open, just as `From` and `To` leave types open. Choose the same list everywhere it occurs in this signature. For `human`, it is `Console`; for `better`, it can be empty. `each` may ask whatever the function it uses may ask.

An undeclared capitalised name used as an entry in an effect list stands for a whole list of effects. It can be empty or contain several effects. We can put known effects alongside it, as in `{Console, Asks}`. Their order doesn't matter. If the same effect occurs more than once, with the same type arguments, it counts once; different type arguments for the same effect name cannot be combined. These are descriptions, not extra inputs to the function.

9. Handlers

The program can answer a request itself. A **handler** goes around an expression and says how to answer:

```
handle better(read()) with
      (answer) ↦ answer
    | (read(), next) ↦ next(rock)
```

The first case receives the final value and gives it back. The second receives the request. It answers rock, then continues.

`next` is what remains to be done, written as a function. Here, what remains is to use `better` on the answer. It also keeps the same handler around that work:

```
(hand) ↦ handle better(hand) with
              (answer) ↦ answer
            | (read(), next) ↦ next(rock)
```

So `next(rock)` continues with rock, `better(rock)` becomes paper, and the handler's first case gives back paper. The whole handled expression comes to `paper`.

With a different answer, the same expression behaves differently. A handler using `next(paper)` would make `better(read())` come to scissors. The expression asks; the handler chooses how to answer.

There are two kinds of handler case:

```
h ::= (p) ↦ e                    for the final value
      (r(p, …), x) ↦ e           for a request, with a variable for the rest
```

`h` stands for a handler case. Its patterns name what the request was given, and the extra variable names the rest. Those names belong to that case's body.

Which handler answers?

A request goes to the nearest surrounding handler with a case for its name. A handler with no such case lets it pass:

```
handle (handle better(read()) with (hand) ↦ worse(hand)) with
      (answer) ↦ answer
    | (read(), next) ↦ next(rock)

    comes to   rock
```

The inner handler has no case for `read`, so the outer one answers rock. Continuing still includes the inner handler: `better(rock)` gives paper, and the inner handler applies `worse` to paper, giving rock.

If no surrounding handler answers, the request reaches the outside, as it did when a person chose a hand. The outside answers once and the program continues.

A handler's case runs outside the handler it just used. Calling `next` puts that handler back around the remaining work. This matters if the case makes another request:

```
handle better(read()) with
      (answer) ↦ answer
    | (read(), next) ↦ next(read())
```

The `read()` in the case asks outside this handler. Once it receives a hand, `next` carries on with it. It doesn't repeatedly handle its own request.

Having a case for a request's name is different from matching its inputs:

```
handle show(win) with
      (answer) ↦ answer
    | (show(lose), next) ↦ next(unit)
```

This gets stuck. The handler takes responsibility for `show`, but its only pattern doesn't match win. It doesn't pass the request on. Likewise, `handle paper with (rock) ↦ scissors` gets stuck because its final value matches no value case.

10. Choosing how to continue

The handler holds `next` as a function. It can use that function once, several times, later, or never.

Stopping

A player can run out of prepared hands:

```
effect Forfeit = forfeit : () → Hand

firstOf : (List(Hand)) → Hand ! {Forfeit}
firstOf = (cons(a, rest)) ↦ a
        | (empty) ↦ forfeit()
```

The handler can make that a loss:

```
handle result(firstOf(empty), rock) with
      (outcome) ↦ outcome
    | (forfeit(), next) ↦ lose

    comes to   lose
```

The case never applies `next`, so the round is never compared. The handler's answer becomes the result of the whole handled expression. An error that abandons the current computation works this way.

Trying several answers

Which hands would beat rock? We can answer `read()` three times and count the wins:

```
handle result(read(), rock) with
      (outcome) ↦ points(outcome)
    | (read(), next) ↦ add(next(rock), add(next(paper), next(scissors)))

    comes to   succ(zero)
```

Each call to `next` starts from the same waiting point, with a different hand. Each result passes through the handler's first case, so each becomes a number: zero for rock, one for paper, zero for scissors.

Here `next` has type `(Hand) → Nat`. The request still needs a hand as its answer, but the handler turns the final outcome into a number. The type makes both ends visible.

This is search: try possible answers and combine what they lead to. We used ordinary functions and data to say what to do with the alternatives.

11. Keeping a score

A score changes as rounds are played. We can describe each change with a score before and a score after:

```
recordWith : (Outcome, Nat) → Pair(Outcome, Nat)
recordWith = (outcome, total) ↦ pair(outcome, add(points(outcome), total))

recordWith(win, zero)
    comes to   pair(win, succ(zero))
recordWith(draw, succ(zero))
    comes to   pair(draw, succ(zero))
```

To record two rounds, we take the score from the first result and pass it into the second call:

```
recordBothWith : (Round, Round, Nat) → Pair(List(Outcome), Nat)
recordBothWith = (first, second, total) ↦
    ((pair(a, afterFirst)) ↦
        ((pair(b, afterSecond)) ↦
            pair(cons(a, cons(b, empty)), afterSecond))
        (recordWith(judge(second), afterFirst)))
    (recordWith(judge(first), total))
```

Read from the first call: judge the first round and record it with `total`. Call its outcome `a` and its score `afterFirst`. Record the second round with that score, then give back both outcomes and the final score.

```
recordBothWith(round(paper, rock), round(rock, rock), zero)
    comes to   pair(cons(win, cons(draw, empty)), succ(zero))
```

`recordBothWith` only passes the score along. A function that uses it would have to pass the score too. Every function between the place the score starts and the places it changes would need to carry it in and out. The core can say a changing score, but not locally: one small idea changes every function on the way. That is the kind of idea requests are for.

Requests let us mention the score where it is actually used:

```
effect State(Stored) = get : () → Stored
                     | put : (Stored) → Unit
```

An effect can leave types open, as a data type can. `State(Nat)` keeps a number; `State(Hand)` keeps a hand. `get()` asks for the current value. `put(a)` asks to replace it with `a` and is answered with `unit`.

The effect declaration shape is now:

```
effect F(X, …) = r : (A, …) → B | …
```

The types of the requests can use the effect's type variables, which follow the same rules as a data type's. An effect with no type variables is written without parentheses, like `Console`.

In one effect list, each effect name has one choice of type arguments. Combining `State(Nat)` and `State(Hand)` isn't allowed: both use the same `get` and `put` names, so their requests couldn't be distinguished.

To do one thing, then return another:

```
then : (Unit, Item) → Item
then = (u, a) ↦ a
```

Inputs are worked out from left to right. The first input therefore happens before the second, even though the body uses only the second. We can record a win by getting the score, adding one, and putting it back:

```
record : (Outcome) → Outcome ! {State(Nat)}
record = (win) ↦ then(put(succ(get())), win)
       | (outcome) ↦ outcome
```

And record two rounds:

```
recordBoth : (Round, Round) → List(Outcome) ! {State(Nat)}
recordBoth = (first, second) ↦
    cons(record(judge(first)), cons(record(judge(second)), empty))
```

`recordBoth` passes no score. Its type says it may ask for state because `record` does. `then` itself is a pure function: the requests in its inputs happen before its body runs.

Answering the memory requests

A handler supplies the score. It passes the current score from one request to the next, but writes that passing in one place:

```
keep : (() → Result ! {State(Stored), Asks}, Stored) → Pair(Result, Stored) ! {Asks}
keep = (task, start) ↦
    (handle task() with
          (answer) ↦ ((current) ↦ pair(answer, current))
        | (get(), next) ↦ ((current) ↦ next(current)(current))
        | (put(value), next) ↦ ((current) ↦ next(unit)(value)))(start)
```

The signature says: give `keep` a task and an initial stored value; it gives back the task's result together with the final stored value. It answers the state requests itself. Other requests, described by `Asks`, pass through it.

The task is a function with no inputs. That keeps its work waiting until `task()` runs inside the handler. Given the work directly, we would run it before the handler was there.

Every case returns a function waiting for the current score:

- At a final value: pair that value with the score.
- At `get()`: answer with the score and continue with the same score.
- At `put(value)`: answer `unit` and continue with `value` as the score.

The `(start)` at the end supplies the first score; the parentheses around the handler keep it out of the last case's body. The two applications in `next(current)(current)` have different jobs. The first answers the request. The second supplies the score to the function returned by the continued handler.

```
keep(() ↦ recordBoth(round(paper, rock), round(rock, rock)), zero)
    comes to   pair(cons(win, cons(draw, empty)), succ(zero))
```

The first round wins and increases the score. The second draws and leaves it alone. The task gives back the outcomes; `keep` adds the final score.

Order now matters in a visible way:

```
keep(() ↦ then(put(succ(zero)), get()), zero)
    comes to   pair(succ(zero), succ(zero))

keep(() ↦ ((saved) ↦ then(put(succ(zero)), saved))(get()), zero)
    comes to   pair(zero, succ(zero))
```

In the first, we put one and then get one. In the second, we get zero first, call it `saved`, then put one. The final score is the same, but the task returns the value it read earlier.

The handler never inspects the stored value. The same definition keeps a hand:

```
keep(() ↦ then(put(paper), get()), rock)
    comes to   pair(paper, paper)
```

The task can also ask to show its outcome:

```
keep(() ↦ then(show(record(win)), get()), zero)
```

`keep` answers the score requests. `show(win)` passes through it to the outside. Once that is answered with `unit`, the expression comes to `pair(succ(zero), succ(zero))`. Here `Stored` is `Nat`, `Result` is `Nat`, and `Asks` is `Console`.

Different effects can appear in the same expression. Each request still finds the nearest handler with a case for its name:

```
handle (handle then(show(win), get()) with
              (answer) ↦ answer
            | (show(outcome), next) ↦ next(unit)) with
      (answer) ↦ answer
    | (get(), next) ↦ next(zero)

    comes to   zero
```

The inner handler answers `show`. The request for `get` passes through it to the outer handler. The result is the number supplied there.

Two scores, two places

One `keep` supplies one store. Two nested handlers for the same `get` and `put` names don't give us two stores we can choose between: the nearer handler answers both names. To keep a score for each player, give each score a **place**, and include that place in each request:

```
Place = mine | yours

effect Scores = look : (Place) → Nat
              | change : (Place, Nat) → Unit
```

`look(mine)` asks for my score; `change(yours, zero)` asks to set yours to zero. Adding a point works for either place:

```
bump : (Place) → Unit ! {Scores}
bump = (place) ↦ change(place, succ(look(place)))
```

The handler keeps both scores. Each case returns a function waiting for the two current values, `a` and `b`:

```
keepScores : (() → Result ! {Scores, Asks}, Nat, Nat) → Result ! {Asks}
keepScores = (task, first, second) ↦
    (handle task() with
          (answer) ↦ ((a, b) ↦ answer)
        | (look(mine), next) ↦ ((a, b) ↦ next(a)(a, b))
        | (look(yours), next) ↦ ((a, b) ↦ next(b)(a, b))
        | (change(mine, value), next) ↦ ((a, b) ↦ next(unit)(value, b))
        | (change(yours, value), next) ↦ ((a, b) ↦ next(unit)(a, value)))(first, second)
```

Looking answers with the selected score and keeps both scores as they were. Changing answers `unit` and replaces only the selected score. At a final value, the handler gives back the task's answer.

```
keepScores(() ↦ then(bump(mine), pair(look(mine), look(yours))), zero, zero)
    comes to   pair(succ(zero), zero)
```

A place is a value, so we can pass it to a function or hold it in data. Two names can refer to the same place:

```
keepScores(() ↦ ((samePlace) ↦ then(bump(samePlace), look(mine)))(mine), zero, zero)
    comes to   succ(zero)
```

`samePlace` was given `mine`. Adding through `samePlace` changes what looking through `mine` finds: both refer to the same score. The scores live in the handler; a place tells it which score we mean.

This uses the same requests, patterns, functions and data as before. Here the two places are fixed. More places can be represented with numbers and their stored values kept in a list.

12. Continuing later

A handler can return the rest of the work instead of carrying it out immediately. Then whoever receives it decides when to continue.

Let a player give its hands one at a time:

```
effect Give = give : (Hand) → Unit

cycle : (Hand) → Unit ! {Give}
cycle = (a) ↦ then(give(a), cycle(better(a)))
```

Starting with rock, it gives rock, paper, scissors, then rock again. It keeps asking forever.

A handler can stop at each hand and return that hand with the rest:

```
Moves = done | gave(Hand, (Unit) → Moves)

moves : (() → Unit ! {Give}) → (Unit) → Moves
moves = (task) ↦ (u) ↦
    handle task() with
          (answer) ↦ done
        | (give(a), next) ↦ gave(a, next)
```

`moves` gives back a function waiting for `unit`. When applied, it starts the task. At `give(a)`, it returns `a` together with `next`, without applying `next`.

The returned `next` also waits for `unit`, the answer to `give`. Applying it reaches the next hand. In these lines, `next`, `after` and `again` name the functions returned alongside the hands:

```
moves(() ↦ cycle(rock))(unit)       comes to   gave(rock, next)
next(unit)                         comes to   gave(paper, after)
after(unit)                        comes to   gave(scissors, again)
```

The player is infinite, but each request for a move reaches one hand. This is a generator: its caller asks for each next item when it needs it.

`Moves` also shows a recursive type containing a function. A `gave` holds a hand and a function that can produce more `Moves`. Such a sequence can keep unfolding indefinitely; it needn't eventually reach `done`.

A recursive type is defined in terms of itself, so the definition has to settle. For `Moves` it does: the more values count as `Moves`, the more functions count as `(Unit) → Moves`, so the two grow together. Compare `Test = test((Test) → Outcome)`. The more values count as `Tests`, the more inputs a `(Test) → Outcome` must accept without getting stuck, so fewer functions count, and fewer values are `Tests`. Counting more makes fewer, and the definition never settles.

So a recursive occurrence may be in data contents or in a function's result, but not in a function's input. That rule applies through other type declarations too. Request-answer types count as inputs supplied from outside, so recursive paths through them are excluded as well.

Taking turns

Two generators can play each other. `turns` asks each for a hand, and `joinMoves` builds a round if both supplied one:

```
turns : (Nat, (Unit) → Moves, (Unit) → Moves) → List(Round)
turns = (zero, first, second) ↦ empty
      | (succ(left), first, second) ↦ joinMoves(left, first(unit), second(unit))

joinMoves : (Nat, Moves, Moves) → List(Round)
joinMoves = (left, gave(a, first), gave(b, second)) ↦
                cons(round(a, b), turns(left, first, second))
          | (left, first, second) ↦ empty
```

The new functions use no new forms. `first` and `second` hold what remains of each player, and `turns` chooses when to continue each one.

```
rocks : () → Unit ! {Give}
rocks = () ↦ then(give(rock), rocks())

turns(succ(succ(succ(zero))), moves(() ↦ cycle(rock)), moves(rocks))
    comes to   cons(round(rock, rock),
                    cons(round(paper, rock),
                         cons(round(scissors, rock), empty)))
```

Each player describes its own moves. `turns` controls their order. This gives a small example of cooperative scheduling: tasks give control back at requests, and a scheduler decides which task continues.

13. The whole medium

Here are the pieces together. The letters stand for parts of the language:

Letter	What it stands for
`e`, `v`	An expression, a value
`p`, `h`	A pattern, a handler case
`x`, `n`	A variable, a defined function's name
`c`, `r`	A constructor, a request
`T`, `F`	A declared type's name, an effect's name
`A`, `B`, `X`	Types, and a variable for a type
`E`, `S`	An entry in an effect list, and a variable for a list of effects

`e, …` means any number of expressions, including none, and likewise for the other letters. `| …` means one or more alternatives. A declared type or effect with no type arguments is written without parentheses; so is a constructor with no contents.

Declarations and definitions

```
T(X, …) = c(A, …) | …                 a type and its constructors
effect F(X, …) = r : (A, …) → B | …   an effect and its requests

n : A                                 a function's type, when written
n = (p, …) ↦ e | …                    its definition
```

A program gives declarations and definitions, then an expression to work out. A signature describes a function's inputs, results and possible requests. It can describe a particular use of a more general function. Omitting it doesn't change evaluation. Writing it doesn't make it true either: whether a function fits its signature is a claim that can be checked, and checking is outside the medium.

Expressions, values and patterns

```
e ::= x                              a variable
      c(e, …)                        data
      n                              a defined function
      (p, …) ↦ e | …                 a function written out
      e(e, …)                        an application
      r(e, …)                        a request
      handle e with h | …            a handled expression

v ::= c(v, …)                        data whose contents are values
      n                              a defined function
      (p, …) ↦ e | …                 a function written out

p ::= x                              match any value and name it
      c(p, …)                        match a constructor and its contents

h ::= (p) ↦ e                        for a final value
      (r(p, …), x) ↦ e               for a request, naming the rest as x
```

Every place marked `e` holds an expression. A handler can be in a function body, as in `moves`; an application can use a handler's result, as in `keep`; and a handler can surround another handler. Functions hold work back, so a request inside a function is made only when that body is used.

Types

```
A ::= T(A, …)                        a declared type, filled with types
      X                              a type variable
      (A, …) → B ! {E, …}            a function type

E ::= F(A, …)                        a declared effect, filled with types
      S                              a variable for a list of effects
```

Parentheses can group any type: `(A)` means the same type as `A`. Leave out `! {…}` when there are no effects. Each effect suffix belongs to the nearest preceding arrow in the same group. Without grouping, the result after an arrow extends to the right.

Type variables in a declaration belong to that declaration. Their names are distinct and cannot reuse declared type or effect names. Every type name used in a declaration must name a declared type or one of that declaration's type variables; every effect named there must be declared. A type or effect takes as many type arguments as its declaration lists.

In a signature, an undeclared capitalised name in a type position stands for an arbitrary type. Used as a whole entry in an effect list, it stands for an arbitrary list of effects. One name cannot stand for both. These choices belong to that signature, and each name has the same choice throughout it.

An effect-list variable can stand for no effects, one, or several. Fill it with its chosen list alongside any named effects. Order doesn't matter, and repeated effects with the same type arguments count once.

Each effect name must have a single choice of type arguments in the combined list. Effect-list variables occur only in signatures.

A value belongs to a declared type when it is made with one of that type's constructors and its contents belong to the types declared for them. Fill the declaration's type variables with the types supplied to it.

A function belongs to a function type when it takes the indicated number of inputs and, given values of the indicated types, behaves as follows however its requests are answered, provided each answer has the type its request declares: it never gets stuck, every request it leaves unanswered belongs to a listed effect and has inputs of the declared types, and any final result belongs to `B`. The computation may also continue forever. For an effect that takes types, read its requests with the listed type arguments filled in: under `State(Nat)`, `put` takes a `Nat` and `get` is answered with one.

A function fits a signature with variables when it belongs to the type made by every choice of them, for types and effect lists alike. `size` fits `(List(Item)) → Nat` because it belongs to `(List(Hand)) → Nat`, `(List(Round)) → Nat`, and every other choice.

Recursive types include all values consistent with these descriptions, including values that keep unfolding through functions. Following a recursive path through declarations must never put the recursive type in a function input or a request-answer type: there, counting more values would make fewer values count, and the definition would never settle. This also applies when the path goes through a type argument in another declaration.

Types describe values. They do not themselves take evaluation steps, and type signatures don't add steps to programs.

Names and grouping

Each global name has one declaration or definition, and a defined function has at most one signature. Each constructor belongs to one type declaration, and each request to one effect declaration. Types and effects, including their type variables, start with capitals; names used in expressions start without capitals.

A definition can use any defined name, including itself or a name defined later. A type declaration can refer to itself or other declared types, subject to the recursive-type restriction above.

In a pattern, a declared constructor means that constructor. Any other name is a variable. The variables of one case are distinct, including the variable for the rest in a request case. They cannot reuse a constructor, request or defined function's name.

A variable is in scope in its case's body, including functions inside it. The nearest case that binds a name determines what the name means. A name in an expression with no such binding must be declared or defined.

A constructor or request is written with its declared number of contents or inputs, also in patterns. A defined name written directly with inputs is given the number its function takes. All cases of a function have that same number of patterns. When applying a value obtained through an expression, a mismatch gets stuck.

Parentheses group an expression or a type. A case body, in a function or a handler, extends as far to the right as it can; use parentheses to end it before applying the function or handler, or placing it in a larger expression. The result after a type arrow also extends to the right unless parentheses group it.

Working out an expression

In an application, first work out what is being applied, then its inputs from left to right. In data or a request, work out the contents or inputs from left to right. In a handler, work on the expression it surrounds. Nothing happens inside a function until it is applied.

Applying a function to values takes one step. Find the first case whose patterns all match. The application becomes that case's body, with each matched value put wherever its variable means it. A defined function uses the cases of its definition in the same way.

Matching follows the shape of data. A constructor pattern needs the same constructor, with each content matching its pattern. A variable matches the whole value at that place. If no case matches, applying gets stuck. Applying data also gets stuck.

When a handled expression becomes a value, the handler takes one step to the body of its first matching value case. Put the matched values in place in that body. With no matching value case, it gets stuck.

When the next work is a request with value inputs, find the nearest surrounding handler with a case for that request's name. The handler takes one step to the body of its first matching request case. Put the inputs' matched values in place, and give the extra variable the rest of the work. If the handler names the request but none of its patterns matches, it gets stuck.

The rest is a function taking one answer. Its body is the interrupted expression with that answer where the request was, surrounded by the same handler. Intervening handlers that let the request pass are kept too. Use a fresh name for the answer so it cannot interfere with any existing variable. The request case itself runs outside the handler just used.

With no handler for a request, evaluation waits for the outside. Its answer replaces the request, and evaluation continues. A whole computation therefore has four possibilities: it returns a value, gets stuck, waits for an answer, or continues forever.

The shape to remember

Piece	How it is made	How it is used
Data	A constructor holds values	A pattern takes them apart
A function	Cases connect patterns to bodies	Application supplies inputs
A request	An operation asks with inputs	A handler supplies a response and controls continuation

Names let functions refer to themselves and to one another. Types make the kinds of contents, inputs and results visible. Every larger example in the book combines these pieces: a list repeats a data

shape, a match repeats a computation, state passes a value between requests, and a generator keeps the rest of a computation for later.

Why these pieces

Functions alone could already compute anything; even hands and numbers can be written as functions. The core gives data its own form anyway, so that it can be seen.

Beyond the core, a form is added only when an idea can't be said locally with what is already there. Saying alone can't wait for an answer from outside, and it could say a changing score only by passing it through every function on the way. So requests and handlers bring doing back on top of saying. Nothing else needed a form of its own: errors, search, state, generators and taking turns are all written *in* the medium with those pieces, not added *to* it.

Types are on a different axis. They say what kinds of things there are, and don't change what any program does.

So when you meet something new, whether a feature of another language or a precise idea of your own, ask what data, what functions and what requests it involves, what kinds of things they are, and how they are combined.